初识卷积神经网络

一张RGB图片是如何被处理成训练数据的？张量是什么？

一张RGB图片在计算机里，本质上是一个三维数组(heigh,width,channel = 3),即数组[3(RGB)][height][width],数组中每个元素表示该通道下该像素点的颜色深浅，三个通路叠起来，三原色就可以合成各种颜色形成图片 R:红色 G:绿色 B:蓝色

处理过程

1. 读取：使用 PIL 或 cv2 读取后，通常得到一个形状为 (h,w,3)
2. 尺寸统一：因为全连接层要求固定输入，必须使用 Resize 把所有图片缩放到 224x224。
3. 归一化：将像素值从 0-255 映射到 0-1 或 [-1, 1]。这能加速模型收敛。常用做法是减去均值再除以标准差。
4. 维度转换(HWC -> CHW)：PyTorch 要求通道维在前。把(h, w, 3)转成 (3, h, w)。
5. 批处理（Batching）：模型训练通常一次输入多张图片。于是形状变成 (Batch_Size, 3, 224, 224)，也就是四位数组

什么是张量（Tensor）？

张量：数学上的定义比较复杂(看不懂)，说是类似一个函数，输入值为向量和余向量(一个函数，输入值为向量，输出值为数，并且是线性的)，输出值为一个数，并且对每个单独的输入值是线性的 线性$f(ax+by)=af(x)+bf(y)$ 但在机器学习里可以理解为3维及以上数组

卷积神经网络分为哪几层？

一个标准的CNN由以下几层交替堆叠而成：

1. 输入层：接收预处理后的图片张量。
2. 卷积层：提取局部特征。
3. 激活层：引入非线性。
4. 池化层：下采样，降低空间尺寸，减少计算量。
5. 全连接层：位于网络尾部，将提取的特征映射到类别空间。
6. 输出层：通常也是全连接层，输出每个类别的得分。

什么是卷积？数学卷积和CNN卷积有区别吗？卷积核、步长是什么？

数学卷积： 卷积是对信号进行翻转，平移，相乘，积分的过程。公式为：

$$(f*g)(t)=\int f(\tau)g(t-\tau)\,d\tau$$

描述系统对输入信号的响应。 CNN卷积： CNN卷积没有翻转步骤。它只是把卷积核放在输入图像上，对应位置相乘(可选偏置)再求和，严格来说这叫互相关。 卷积公式：输出尺寸 $=\frac{\text{输入尺寸}-\text{卷积核}*\text{空置}+2*Padding}{\text{步长}}+1$ 卷积核： 一个小的权重矩阵。它像一个滑动窗口，在图像上滑动，步长可调，提取局部特征。不同的卷积核学习到不同的特征（比如有的识别横线，有的识别竖线）。

步长： 卷积核每次滑动的距离。 Stride = 1：逐个像素滑动，输出尺寸较大。 Stride = 2：隔一个像素滑动，输出尺寸减半（下采样）。

池化有哪些类型？作用是什么？感受野是什么？

池化的类型与作用：

和卷积差不多，有一个池化核，步长不设定的情况下步长为池化核大小，只不过具体运算法则不同 最大池化：取窗口内的最大值。作用是保留最显著的特征（比如最亮的边缘），同时降维。 平均池化：取窗口内的平均值。作用是保留整体背景信息，平滑特征。 全局平均池化：将整个特征图压缩成一个值，常用于替代全连接层，减少参数量。

池化的作用：降维，扩大感受野，提供一定的平移不变性(图片平移，但结果相似)，防止过拟合。

感受野：

指特征图上的一个像素点，对应到原始图像上的区域大小。 感受野的大小与神经网络的深浅相关

激活层的作用？CNN最后输出什么？

激活层的作用

引入非线性。如果没有激活函数，无论多少层卷积，本质上都等价于一次线性变换，模型无法拟合复杂函数。常用的激活函数是ReLU=max(0,x)，计算简单，能缓解梯度消失问题。

CNN最后输出什么？

1. 训练阶段：输出未归一化的得分。
2. 推理阶段：在最后接一个Softmax 函数，把得分转换成概率（总和为 1）。得分最高的那个索引，就是模型预测的花卉类别。

炼丹

什么是超参数？如何设定？

什么是超参数？

在开始训练之前，手动设定的参数。它们不参与反向传播的更新。 如：学习率，批次大小，迭代次数，卷积核大小卷积核数量，层数，Dropout丢弃率，优化器选择

3. 如何设定？

使用默认值：学习率=0.001，Batch Size=32/64，Epochs=50 尝试：学习率尝试 [0.1, 0.01, 0.001]，Batch Size尝试[32, 64]，排列组合挨个试。 随机：在给定的范围内随机采样超参数 学习率调度器：不要固定学习率。前期设大一点快速收敛，后期调小微调 自动化调参工具：Optuna、Ray Tune

什么是优化器？有哪些类型？

什么是优化器

寻找损失函数的最小值时，根据模型参数，优化器反向传播算出的梯度，决定更新哪些参数与更新多少，从而更新网络权重。

常见的优化器类型：

SGD：每次选择一部分参数更新 SGD + 动量：保持之前的一部分更新大小 Adam：为每个参数动态调整学习率(频繁更新的参数步子小一点，不常更新的步子大一点)，并且结合了动量。

AdamW：Adam改进版，修正了权重衰减方式，让正则化不被算入到动态调整学习率的过程中，避免高频修正的参数正则化力度小的缺陷。

RMSprop：一种自适应学习率优化器，适合处理非平稳目标，常用于 RNN。

DataLoader 与 Dataset 是什么及其使用?

在PyTorch中，数据的读取和预处理由 torch.utils.data负责。

1. Dataset（数据集）

一个抽象类，代表整个数据集。包含数据位置，数量，读取方式

```
class flower(Dataset):
    def __init__(self,root_dir,label_dir):
        self.root_dir = root_dir
        self.label_dir = label_dir
        self.path = os.path.join(self.root_dir,self.label_dir)
        self.img_path = os.listdir(self.path)

    def __getitem__(self,idx):
        img_name = self.img_path[idx]
        img_item_path = os.psth.join(self.root_dir,self.label_dir,img_name)
        img = Image.open(img_item_path)
        label = self.label_dir
        return img,label

    def __len__(self):
        return len(self.img_path)
```

2. DataLoader（数据加载器） 从Dataset中加载数据，打乱顺序(可选)并打包 `train_loader = DataLoader(dataset = train,batch_size = 32,shuffle = True,generator=g,drop_last = True)`

模型的优化与微调

模型层数越多、结构越深，性能就一定越好吗?

否 模型层数越高，会导致参数过多，优化困难，深层优化不动（退化问题），出现梯度消失/梯度爆炸问题，导致模型易过拟合且泛化能力不强

ResNet与VGG

用了二者的与训练参数 幸好在运行vgg前将代码给ai看了一遍，参数量太大，显存差点爆了，所以冻结了卷积层并减小了包 ResNet50 **VS** VGG16 最终结果来看，虽然vgg体量更大，但是正确率比ResNet50差了2~3%，推测原因是因为大体量模型在小数据集上面不匹配，反而小模型更占优

模型的保存与载入

`torch.save(model, save_path)`与`model.load(torch.load(save_path, map_location=device))` `map_location=device`参数告诉模型用device对应的设备来跑模型，避免了因为训练使用GPU而实际使用时用CPU导致报错 不过AI推荐在model处写`model.state_dict()`,加载时用`model.load_state_dict()`，以字典保存，可移植性和占用空间更小

模型训练

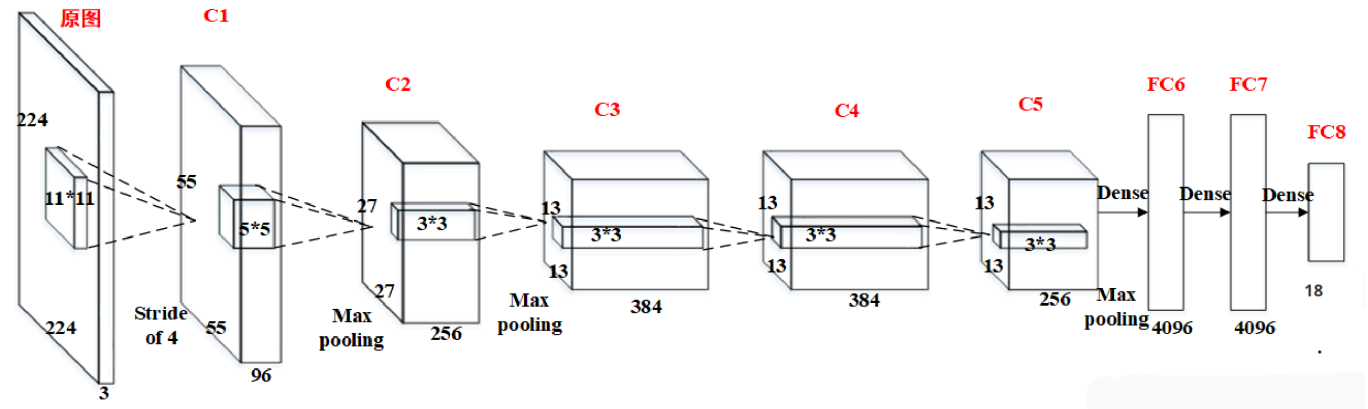
数据预处理

数据处理源代码在src/data_processing.py里

1. 删去损坏数据
2. 划分训练集，验证集和测试集
3. 数据增强(具体用什么操作————翻转，平移，放缩，错切，亮度等————问了AI) transforms

模型搭建

任务要求模型图片解析



1. C1：用11x11的卷积核处理224x224的3通道图片，步长为4，提取出55x55的96通道特征图,升维
2. Max pooling：最大池化，下采样，减少特征维度
3. C2：用5x5的卷积核处理55x55的96通道特征图，提取出27x27的256通道特征图，升维
4. Max pooling
5. C3：用3x3的卷积核处理27x27的256通道特征图，提取出13x13的384通道特征图，升维
6. C4：用3x3的卷积核处理13x13的384通道特征图，提取出13x13的384通道特征图，增加层数，保持维度
7. C4：用3x3的卷积核处理13x13的384通道特征图，提取出13x13的256通道特征图，降维

8. Max pooling
9. 展平为13*13*256=43264个神经元
10. FC6：全连接层，本质为线性层，输入43264个数值，输出4096个神经元
11. FC7：全连接层，增加层数，保持神经元个数
12. FC8：输入4096个神经元，输出18(在这里不是只有5个分类吗？最开始写18时AI说我写错了)个神经元，输出层

dataset与dataloader

ps：最初跟着土堆的视频自己写了dataset，但后来ai说可以使用pytorch内建的ImageFolder功能，直接省掉了dataset

超参数

1. 模型如上图
2. 损失函数：交叉熵损失
3. 优化器：Adam
4. 学习率调度器：阶梯式学习率衰减，初始学习率1e-4，每训练7个epoch，把学习率乘以0.5
5. dropout：0.3
6. 早停
7. L2正则化：1e-4

使用GPU进行训练

ps： 最初运行时报错，发现是pytorch和cuda不兼容，所以下了新版本的pytorch

最开始重复打印了

```
使用设备: cuda
数据目录: project/data_split/flower 存在: True
```

这一串东西，ai解释说是DataLoader的num_workers=2开多进程时会反复判断，所以就创建了`if __name__ == "__main__"`来避免反复打印

训练过程

第一次发现最后结果准确率极低，发现模型训练轮数太少，后期学习率太低，增加了训练轮数并加入早停,调整了学习率调度器调整学习率的幅度 第二次训练依然不理想，卡在了65%左右就早停了，问ai说是模型太大而训练数据太少，修改了dropout，加入L2正则化，并且减小图片输入尺寸为128x128 第三次准确率上升至70%，AI分析后说是已经到极限了，再改就要更换模型了，说是模型参数太大，而训练集太小